CM5 : Feature Maps

1 Introduction

Les modèles linéaires sont intéressants mais peuvent manquer d'expressivité, que faire pour un dataset non linéairement séparable ou dans un problème de régression, si nos points suivent une loi polynomiale ? Appliquer un modèle linéaire à de telles données est un cas typique d'underfitting, les performances seront mauvaises car le modèle n'est pas assez complexe, l'erreur est alors due au biais. Une solution est d'utiliser des modèles plus complexes, non linéaires, mais il est en réalité possible de d'incorporer la non linéarité à des modèles linéaires.

2 Feature maps

On peut rendre non-linéaire un modèle linéaire en créant de nouveaux attributs (features) à partir des attributs originaux de nos données : on fabrique de nouveaux attributs en combinant les précédents. On a une fonction non linéaire, nommée **feature map**, des features vers un nouvel espace de features. Formellement, pour une donnée $\vec{x} \in \mathbb{R}^D$, on applique une transformation $\vec{x} \mapsto \phi(\vec{x})$. La fonction ϕ est la feature map et $\phi(\vec{x}) \in \mathbb{R}^{D'}$ avec $D' > D$ ce qui permet de capturer des interactions non linéaires de nos attributs originaux.

On peut essayer de construire à la main notre feature map, par exemple pour XOR en ajoutant la feature x_1x_2. De façon plus générale, on peut construire tous les attributs issus de combinaisons d'ordre P d'un ensemble de D attributs en combinant toutes les features entre elles, c'est la feature map polynomiale (aussi appelé kernel polynomial). Par exemple, avec $D = 3$ dimensions, pour l'ordre $P = 2$, (notre feature map comprend toutes les combinaisons linéaires de deux attributs), on obtient :

$$\vec{\phi}(\vec{x}) = (1, \sqrt{2}x_1, \sqrt{2}x_2, \sqrt{2}x_3, \sqrt{2}x_1x_2, \sqrt{2}x_1x_3, \sqrt{2}x_2x_3, x_1^2, x_2^2, x_3^2)$$

Quelle est la dimension des données après application de cette feature map ?

Dans le cas général, quelle est la dimension des données après application d'une feature map polynomiale d'ordre P sur des données de dimension D ?

$$\binom{D+P}{P}$$

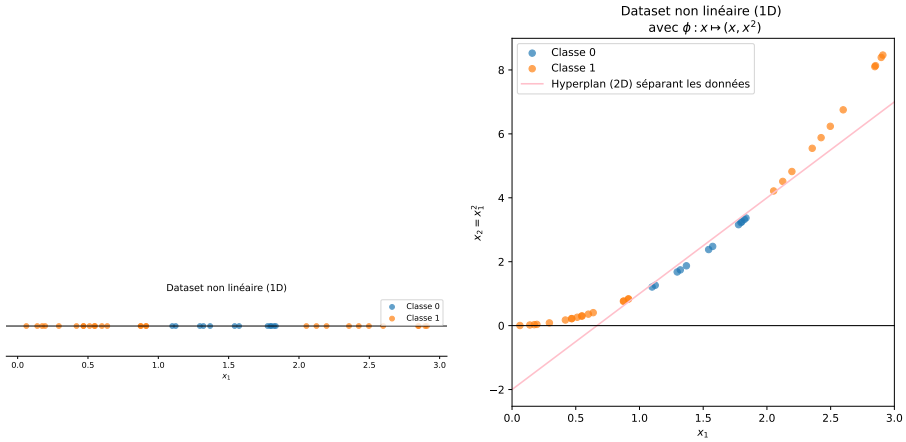

FIGURE 1 – Dataset en 1D non linéairement séparable, l'ajout d'une feature $x_2 = x_1^2$ le rend linéairement séparable.

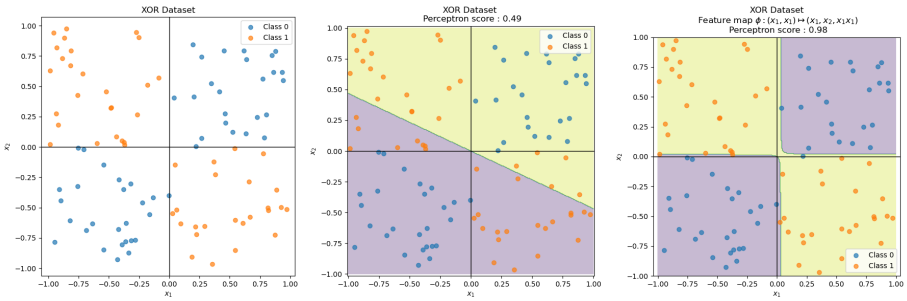

FIGURE 2 – On ajoute la feature x_1x_2 à notre dataset XOR (non linéaire) ce qui permet au perceptron (classificateur linéaire) de le résoudre.

C'est beaucoup, acceptable pour $P = 2$, même avec un grand nombre de dimensions, mais pas bien plus. Heureusement il est possible d'utiliser une feature map sans jamais avoir à effectuer le changement de base, c'est le kernel trick (nous ne développerons pas cette notion pendant le cours).

L'intérêt des features maps est que **la simplicité mathématique des modèles linéaires peut être utilisée** y compris sur des **combinaisons non linéaires** des attributs (features) entre eux (x_1^2 n'est pas linéaire en x_1).

Pour tout problème de classification binaire avec N données, il est possible de fit parfaitement le dataset avec une séparation linéaire en utilisant N features. En quoi est ce un problème ? En ajoutant des dimensions, on facilite l'overfitting sur nos données d'entraînement. Si l'on ajoute plus de features que la taille de nos données, le modèle peut même parfaitement apprendre "par coeur" nos données. En d'autres termes, **ajouter des features permet de réduire le**

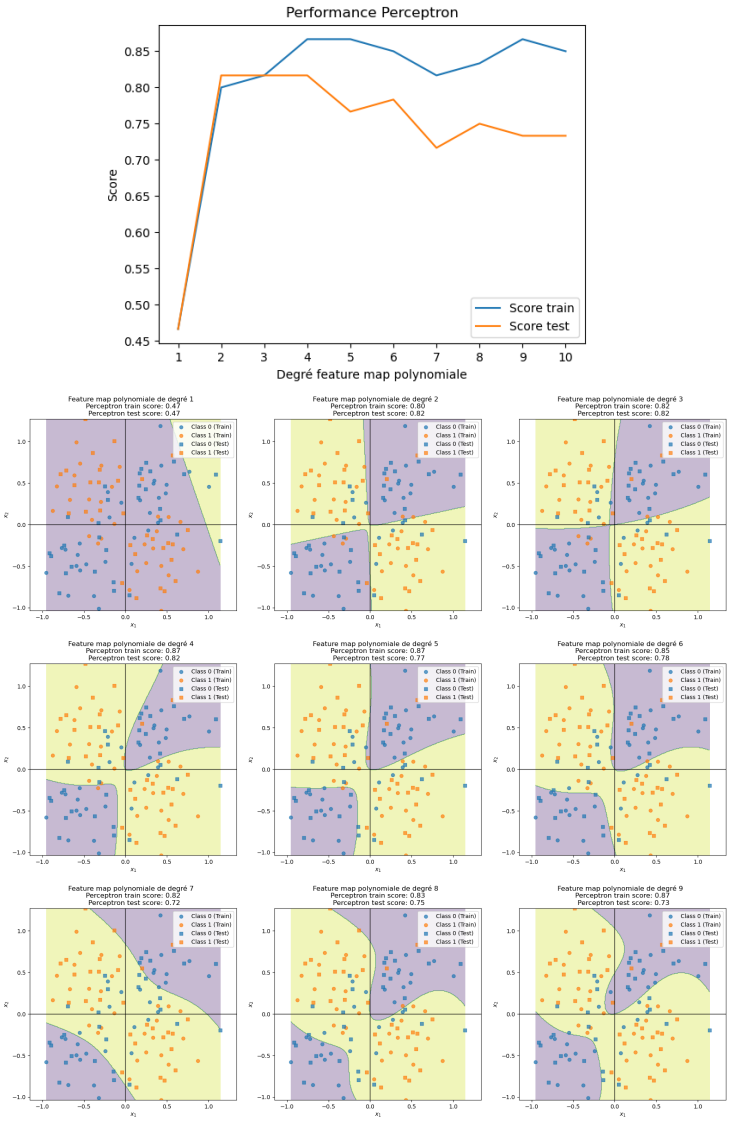

FIGURE 3 – Performance du perceptron sur un dataset XOR bruité en fonction du degré de la feature map polynomiale appliquée aux données. A partir de la courbe de performance, quel est le meilleur choix de degré ? Que se passe t il avec un degré plus faible ? Plus élevé ? Le degré P de la feature map est un **hyperparamètre** du modèle.

biais du modèle, mais augmente sa variance.

3 Bonus : Kernels

Les features maps ont un problème majeur : la dimension des données explose très rapidement à des niveaux dépassant largement les capacités de calcul de n'importe quel ordinateur ou serveur.

En conséquence, pour une feature map ϕ et des données $\vec{x}_i$, $\vec{x}_j$, nous n'allons jamais calculer directement $\phi(\vec{x}_i)$ ou $\phi(\vec{x}_j)$, mais seulement le produit scalaire $\phi(\vec{x}_i).\phi(\vec{x}_j)$. Ce dernier sera noté $K(\vec{x}_i, \vec{x}_j)$. Cette astuce est le **kernel trick**, nous allons maintenant démontrer son bien fondé.

Montrons tout d'abord que lors d'une classification linéaire, le vecteur de poids $\vec{w}$ peut s'écrire comme une combinaison linéaire des données. Nous allons le montrer en prenant pour exemple la loss du perceptron (ce résultat se généralise à d'autres losses) :

$$\mathcal{L} = \frac{1}{N} \sum_{n=1}^{N} \max(0, -y_n(\vec{w}.\vec{x_n}))$$

Et nous en avions calculé le gradient :

$$\vec{\nabla}_\Theta \mathcal{L} = -\frac{1}{N} \sum_{n \in \text{mal classés}} \vec{x_n} y_n$$

La mise à jour du vecteur de poids par descente de gradient était alors :

$$\vec{w} \leftarrow \vec{w} + \eta \frac{1}{N} \sum_{n \in \text{mal classés}} \vec{x_n} y_n$$

Montrons par récurence que à chaque étape $\vec{w}$ peut s'écrire comme une combinaison linéaire des données, c'est à dire $\vec{w} = \sum_i^N \alpha_i x_i$.

Démonstration. **Initialisation**. Une initialisation possible de $\vec{w}$ est simplement $\vec{w}_0 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$ qui est bien une combinaison linéaire des données.

Récurence. Soit $\vec{w} = \sum_n^N \alpha_n \vec{x}_n$ à une étape de la descente de gradient. A l'étape suivante on a :

$$
\begin{aligned}
\vec{w} &\leftarrow \vec{w} + \eta \frac{1}{N} \sum_{n \in \text{mal classés}} \vec{x_n} y_n \\
&= \sum_{n=1}^{N} \alpha_n \vec{x}_n + \eta \frac{1}{N} \sum_{n \in \text{mal classés}} \vec{x_n} y_n \\
&= \sum_{n \in \text{bien classés}}^{N} \alpha_n \vec{x}_n + \sum_{n \in \text{mal classés}} (\alpha_n + \eta \frac{1}{N} y_n) \vec{x_n} \\
&= \sum_{n=1}^{N} \alpha'_n \vec{x}_n \quad \text{avec} \quad \alpha'_n = \begin{cases} \alpha_n & \text{si bien classé} \\ \alpha_n + \dfrac{\eta}{N} y_n & \text{si mal classé} \end{cases}
\end{aligned}
$$

□

Il n'est donc pas nécessaire de calculer directement $\vec{w}$ (dans le cas d'une feature map, ce serait un vecteur bien trop grand), mais seulement les coefficients α_n qui eux sont au nombre de N, la taille de nos données, indépendante de la feature map choisie.

La prédiction de notre modèle peut alors s'écrire :

$$\hat{y}_n = \text{signe}(\vec{w}.\vec{x}_n) = \text{signe}(\sum_{i=1}^{N} \alpha_i \vec{x}_i.\vec{x}_n)$$

Pour calculer la prédiction du modèle, il n'est ainsi pas nécessaire de calculer nos données $\vec{x}_i$, mais seulement la valeur de leurs produit scalaire deux à deux i.e. $\vec{x}_i.\vec{x}_j$ pour tout i, j dans $\{1...N\}$. Il pourrait sembler que pour calculer $\vec{x}_i.\vec{x}_j$ il faille connaitre $\vec{x}_i$ et $\vec{x}_j$, mais non, c'est le kernel trick.

Il existe plusieurs kernels, gardons l'exemple de notre feature map polynomial, d'ordre P, on a :

$$\phi(\vec{x}_i).\phi(\vec{x}_j) = (1 + \vec{x}_i.\vec{x}_j)^P$$

Si calculer $\phi(\vec{x}_i)$ et $\phi(\vec{x}_j)$ demande de l'ordre de D^P opérations [1], calculer $(1 + \vec{x}_i.\vec{x}_j)^P$ demande seulement de l'ordre de $D + P$ opérations. La clef de l'astuce étant que l'on a jamais à calculer explicitement la projection de nos données dans le nouvel espace défini par ϕ pour connaitre leur produit scalaire dans cet espace.

1. $\binom{D+P}{P} \sim D^P$